

Navigation

Թայվանը ունի բարձր լեռնաշղթաների մեծ խտություն՝ ներառելով ավելի քան 250 գագաթ, որոնք բարձր են 3,000 մետրից: A-Ming-ը պլանավորում է կառուցել ռոբոտացված գիպլայն առաքման ցանցեր Թայվանի Կենտրոնական լեռնաշղթայում փաթեթները հեռավոր գյուղերի միջև տեղափոխելու համար:

A-Ming-ի պլանում յուրաքանչյուր ցանց բաղկացած է N հանգույցներից, համարակալված 0-ից մինչև $N - 1$: Այս հանգույցներից **ճիշտ** $K = 6$ -ը էլեկտրակայաններ են, իսկ մնացած $N - K$ հանգույցները՝ գյուղական հանգույցներ: Հանգույցները միացված են $N - 1$ **երկկողմանի** մալուխներով, համարակալված 0-ից մինչև $N - 2$: Յուրաքանչյուր i -ի համար ($0 \leq i \leq N - 2$), i -րդ մալուխը միացնում է $U[i]$ և $V[i]$ հանգույցները: Յուրաքանչյուր մալուխ միացնում է երկու տարբեր հանգույցներ, և յուրաքանչյուր զույգ հանգույցների միջև կա առավելագույնը մեկ մալուխ: Երաշխավորվում է, որ հնարավոր է մալուխների միջոցով անցնել ցանկացած զույգ հանգույցների միջև:

a և b հանգույցների միջև **հեռավորությունը** նշվում է $d(a, b)$ և սահմանվում է հետևյալ կերպ.

- Եթե $a = b$, ապա $d(a, b) = 0$:
- Հակառակ դեպքում, $d(a, b)$ -ը մալուխների նվազագույն քանակն է, որը պետք է անցնել a -ից b հասնելու համար:

Ասում ենք, որ ցանցի **ճյուղավորման գործոնը** առնվազն δ է, եթե ցանցի յուրաքանչյուր հանգույց միացված է կամ ճիշտ 1, կամ առնվազն δ մալուխների: A-Ming-ը գիտի մի դրական ամբողջ B թիվ, որի համար ցանցի ճյուղավորման գործոնը առնվազն B է:

A-Ming-ը պատրաստել է 100 տարբեր տեսակի մալուխներ, համարակալված $0, 1, \dots, 99$: Ցանցի յուրաքանչյուր մալուխ կկառուցվի այդ տեսակներից մեկով:

Ռոբոտները կշարժվեն գիպլայն մալուխներով՝ առաքման առաջադրանքներ կատարելու համար: A-Ming-ը ցանկանում է տեղադրել նավիգացիոն ծրագիր, որը կօգնի ռոբոտներին վերադառնալ էլեկտրակայաններ և լիցքավորվել: Սակայն ռոբոտները ունեն չափազանց սահմանափակ հիշողություն: Ուստի նավիգացիոն ծրագիրը նախագծելիս ռոբոտները կողմնորոշվում են միայն էլեկտրակայանների $K = 6$

քանակով, երաշխավորված ճյուղավորման B գործոնով, և ռոբոտի ներկայիս հանգույցին կցված մալուխների տեսակներով:

Երբ ռոբոտը գտնվում է մի գյուղական u հանգույցում, որը միացված է երկու կամ ավելի մալուխների, այն նախ սկանավորում է u -ին կցված մալուխները կամայական հերթականությամբ: Ծրագրին տրվում է այդ մալուխների տեսակների ցուցակը՝ այդ հերթականությամբ: Յուրաքանչյուր մալուխի համար ծրագիրը պետք է հաշվի, թե քանի էլեկտրակայան կա, որոնց համար այդ մալուխով շարժվելը նվազեցնում է ռոբոտի և տվյալ էլեկտրակայանի միջև հեռավորությունը:

Մասնավորապես, թող $v_1, \dots, v_k$ ($k \geq 2$) լինեն այն հանգույցները, որոնք անմիջապես կապված են u -ին մալուխներով, և c_i ($1 \leq i \leq k$) լինի u և v_i հանգույցները միացնող մալուխի տեսակը: Այդ դեպքում ծրագրին տրվում են K , B , և ցուցակը $c_1, c_2, \dots, c_k$: Յուրաքանչյուր i -ի համար ($1 \leq i \leq k$), ծրագիրը պետք է որոշի էլեկտրակայանների քանակը p , որոնց համար $d(v_i, p) < d(u, p)$:

Ձեր խնդիրը է մշակել ռազմավարություն մալուխների տեսակների նշանակման համար և նախագծել նավիգացիոն ծրագիրը: Ձեր լուծման գնահատականը կախված է օգտագործված մալուխների տարբեր տեսակների քանակից (տես «Ենթախնդիրներ և գնահատում» բաժինը մանրամասների համար):

Իրականացման մանրամասներ

Դուք պետք է իրականացնեք երկու ֆունկցիա, մեկը մալուխների տիպերը որոշելու համար, մյուսը՝ ռոբոտների ծրագրի համար:

Մալուխների տիպերը նշանակելու ֆունկցիան, որ պետք է իրականացնեք, սա է.

```
std::vector<int> construct_network(int N, int K, int B,
                                   std::vector<int> U, std::vector<int> V,
                                   std::vector<int> P)
```

- N . հանգույցների քանակը:
- K . էլեկտրակայանների քանակը:
- B . ցանցի երաշխավորված մինիմալ ճյուղավորման գործոնը:
- U, V . զանգվածներ, $N - 1$ երկարության, նկարագրում են մալուխների միացումները:
- P . զանգված, $K = 6$ երկարության, նկարագրում է էլեկտրակայաններով հանգույցները:
- Այս ֆունկցիան կանչվում է **առավելագույնը 10 000 անգամ** յուրաքանչյուր թեստի համար:

Այս ֆունկցիան պետք է վերադարձնի $N - 1$ երկարությամբ T զանգված.

- $T[i]$ տիպի մալուխը օգտագործվում է $U[i]$ և $V[i]$ հանգույցները միացնելու համար:
- Յուրաքանչյուր $T[i]$ պետք է բավարարի $0 \leq T[i] < 100$ պայմանին:

Ֆունկցիան, որը պետք է իրականացնեք **ռոբոտների ծրագրի** համար հետևյալն է.

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K ՝ էլեկտրակայանների քանակը:
- B ՝ ցանցի երաշխավորված նվազագույն ճյուղավորման գործոնը:
- C ՝ տվյալ **գյուղական հանգույցին, որը միացված է առնվազն 2 մալուխի**, կցված բոլոր մալուխների տեսակների ցուցակը՝ կամայական հերթականությամբ:
- Երաշխավորվում է, որ C -ի երկարությունը առնվազն B է:
- Այս ֆունկցիան կանչվում է առավելագույնը 100 000 անգամ յուրաքանչյուր թեստի համար:
- `navigate` ֆունկցիան չի կարող հիմնվել սկզբնական ցանցի կառուցվածքի վրա. մասնավորապես, գնահատման համակարգը կարող է վերադասավորել `navigate`-ի բոլոր կանչերը նույն թեստի տարբեր կառուցված ցանցերի միջև:

Այս ֆունկցիան պետք է վերադարձնի D զանգված.

D զանգվածի երկարությունը պետք է լինի նույնը, ինչ C զանգվածինը: $D[i]$ -ը պետք է լինի այն էլեկտրակայանների քանակը, որոնց համար $C[i]$ -ին համապատասխան մալուխով շարժվելը նվազեցնում է ռոբոտի և տվյալ էլեկտրակայանի միջև հեռավորությունը:

Նշենք, որ գնահատման համակարգում `construct_network` և `navigate` ֆունկցիաները կանչվում են **երկու առանձին պրոցեսներում**:

Սահմանափակումներ

- $K = 6$.
- $K < N \leq 100\,000$.
- N -երի գումարը բոլոր `construct_network` ֆունկցիաների կանչերում չի գերազանցում 100 000-ը յուրաքանչյուր թեստում:
- $0 \leq U[i] < V[i] < N$, որտեղ $0 \leq i \leq N - 2$.
- Հնարավոր է մալուխների միջոցով ցանկացած հանգույցից գնալ ցանկացած այլ հանգույց:
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- Յուրաքանչյուր C զանգված հանդիսանում է առնվազն 2 մալուխի միացված որևէ գյուղական հանգույցին կցված բոլոր մալուխների տեսակների ցուցակի մի

տեղափոխություն (permutation):

- Յուրաքանչյուր թեստի համար `navigate`-ի բոլոր կանչերի ընթացքում C զանգվածների երկարությունների գումարը չի գերազանցում 200 000:

Ենթախնդիրներ և գնահատում

Ենթախնդիր	Միավոր	Լրացուցիչ սահմանափակումներ
1	6	$B = 2, N = 7$.
2	14	$B = 2, U[i] = i, V[i] = i + 1$ յուրաքանչյուր i համար, $0 \leq i < N - 1$:
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

Ցանկացածի թեստում, եթե հետևյալ պայմաններից գոնե մեկը կատարվում է, ապա այդ թեստի համար ձեր լուծման գնահատականը կլինի 0 (CMS-ում կնշվի որպես `Output isn't correct`).

- `construct_network`-ի որևէ կանչի վերադարձված արժեքը անվավեր է (invalid):
- `navigate`-ի որևէ կանչի վերադարձված արժեքը սխալ է:

Հակառակ դեպքում, թող S -ը լինի ամենափոքր ամբողջ թիվը, որը մեծ է յուրաքանչյուր `construct_network` կանչից վերադարձված յուրաքանչյուր T զանգվածի բոլոր տարրերից: Այլ կերպ ասած, S -ը ամենափոքր ամբողջ թիվն է, որի դեպքում բոլոր կառուցված ցանցերում օգտագործվում են միայն $0, 1, \dots, S - 1$ տիպերի մալուխներ:

Յուրաքանչյուր ենթախնդրի համար ձեր գնահատականը կախված է S -ից՝ հետևյալ կերպ.

Condition	Subtask 1	Subtask 2	Subtask 3 and 4	Subtask 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2\log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

Մասնավորապես, 1, 2 և 5 ենթախնդիրներում կարող եք ստանալ լրիվ միավորը, եթե $S \leq 5$, իսկ 3 և 4 ենթախնդիրներում կարող եք ստանալ լրիվ միավորը, եթե $S \leq 4$:

Օրինակ

Դիտարկենք հետևյալ սցենարը, որտեղ $N = 9$, $K = 6$, and $B = 2$:

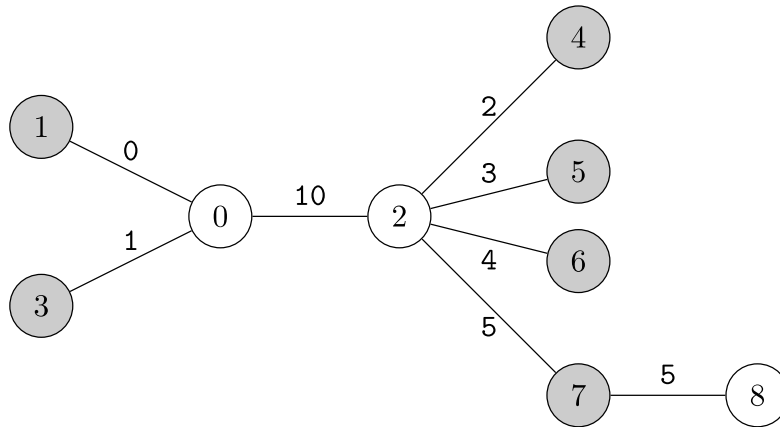
Ցանցի պլանը նկարագրված է $U = [0, 0, 0, 2, 2, 2, 2, 7]$, $V = [1, 2, 3, 4, 5, 6, 7, 8]$:
Էլեկտրակայաններն են $P = [1, 3, 4, 5, 6, 7]$: Դիտարկենք հետևյալ կանչը առաջին պրոցեսում.

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                  [1, 2, 3, 4, 5, 6, 7, 8],
                  [1, 3, 4, 5, 6, 7])
```

Ենթադրանք A-Ming-ը կառուցում է ցանցը և վերադարձնում է.

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

Կառուցված ցանցը պատկերված է ստորև.



Ապա, երկրորդ պրոցեսում ենթադրենք ռոբոտը պետք է կողմնորոշվի (navigate) 0 հանգույցում: 0 հանգույցը միացված է 1, 2 և 3 հանգույցներին: Եթե ռոբոտը կարողում է մալուխների տիպերը այս հերթականությամբ, հետևյալ կանչը պետք է կատարվի.

```
navigate(6, 2, [0, 10, 1])
```

Ցանցի կառուցվածքի հիման վրա

- Առաջին էլեկտրակայանը, հանգույց 1-ը, ամենակարճ հեռավորությունն ունի ինքն իր հետ: Մասնավորապես, $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$:
- Երկրորդ էլեկտրակայանը, հանգույց 3-ը, ամենակարճ հեռավորությունն ունի ինքն իր հետ: Մասնավորապես, $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$:
- Մյուս էլեկտրակայանները, հանգույցներ 4, 5, 6, և 7, ավելի կարճ հեռավորություն ունեն հանգույց 2-ի հետ:
Մասնավորապես, $1 = d(2, u) < 2 = d(0, u) < 3 = d(1, u) = d(3, u)$, որտեղ $u = 4, 5, 6$, or 7:

Էլեկտրակայանների քանակը, որոնց հեռավորությունը փոքրանում է հետյալ կաբելներից հետո (0,1), (0,2) և (0,3), սրանք են՝ 1, 4 և 1, համապատասխանաբար: Հետևաբար, ֆունկցիան պետք է վերադարձնի.

```
[1, 4, 1]
```

Նշենք, որ navigate-ը կարող է նաև կանչվել C -ի տարբեր տեղափոխությունների համար: Օրինակ՝ `navigate(6, 2, [1, 0, 10])` նույնպես հնարավոր կանչ է 0 հանգույցի համար: Այդ դեպքում ֆունկցիան պետք է վերադարձնի [1, 1, 4]:

Ենթադրենք, որ ռոբոտը պետք է կողմնորոշվի (navigate) 2 հանգույցում: Կատարվում է հետևյալ կանչը.

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

Ֆունկցիան պետք է վերադարձնի.

```
[2, 1, 1, 1, 1]
```

Այս ցանցում `navigate` ֆունկցիան **երբեք** չի կանչվի այլ հանգույցների համար, քանի որ.

- 1, 3, 4, 5, 6 և 7 հանգույցները բոլորը էլեկտրակայաններ են, և
- 8 հանգույցը միացված է միայն մեկ մալուխի:

Այս թեստում օգտագործված մալուխների տեսակներն են 0, 1, 2, 3, 4, 5, և 10: Հետևաբար, գնահատականը որոշելու համար կօգտագործվի $S = 11$ արժեքը:

Այս առաջադրանքի կցված փաթեթը, բացի այս օրինակից, պարունակում է նաև երկու այլ օրինակային մուտքեր և ելքեր `sample grader`-ի համար:

Գրեյդերի Նմուշ

Գրեյդերի նմուշը `construct_network` և `navigate` ֆունկցիաները կանչում է նույն պրոցեսի մեջ: Բացի այդ, երբ գրեյդերի նմուշը կանչում է `navigate`, մալուխների տեսակները ներկայացվում են այն հերթականությամբ, որով համապատասխան մալուխները հայտնվում են մուտքում: Այս երկու վարքագծերը տարբերվում են իրական գրեյդերից:

Մուտքային տվյալների ձևաչափը.

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

Յուրաքանչյուր `scenario`-յի ձևաչափըն այսպիսին է.

```

N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]

```

Յուրաքանչյուր scenario-յում, գրեյդերի սմուլն անելու է `navigate` ֆունկցիայի Q կանչ, i -րդ կանչը $X[i - 1]$ հանգույցի վերաբերյալ ինֆորմացիայով: Նկատենք, որ գրեյդերի սմուլը թույլ է տալիս K -ի արժեքը սահմանել նաև ճից տարբեր ամբողջ թվերի:

Ելքի ձևաչափ

Եթե `construct_network` կամ `navigate` ընթացակարգերից որևէ մեկի վերադարձված զանգվածը անվավեր է, գրեյդերի սմուլը դադարեցնում է աշխատանքը և տպում է պատճառը:

Հակառակ դեպքում, գրեյդերի սմուլը յուրաքանչյուր `navigate` կանչից վերադարձված D զանգվածը տպում է առանձին տողում:

Բոլոր սցենարների մշակումից հետո գրեյդերի սմուլը սխալների ստանդարտ հոսքում (`serr`) տպում է հետևյալ տողը.

```
S = S'
```

որտեղ S' -ը ամենափոքր ամբողջ թիվն է, որը մեծ է յուրաքանչյուր `construct_network` կանչից վերադարձված բոլոր T զանգվածներում եղած բոլոր թվերից: